

In the previous lecture we introduced separation logic: heap assertions, the separating conjunction $\star$, small axioms for the heap-manipulating commands, and the frame rule. In this lecture we put the logic to work on examples. We define recursive predicates to describe linked lists in the heap, and we use them to verify two programs: one that frees a list, and one that appends two lists. We then introduce the *magic wand*, a connective that lets us reason backwards about heap commands.

1 Review

Recall the language of heap assertions,

$$H, J, K ::= \mathbf{emp} \mid [P] \mid e_1 \hookrightarrow e_2 \mid H_1 \star H_2 \mid H_1 \bowtie H_2 \mid H_1 \wp H_2 \mid \forall i. H \mid \exists i. H$$

where $H_1 \star H_2$ holds when the heap splits into two disjoint parts satisfying H_1 and H_2 . Recall also that triples are *partial correctness with fault avoidance*: a valid triple $\{H\} c \{J\}$ guarantees that c never produces a memory error from a state satisfying H , and that if c terminates, the final state satisfies J . The small axioms for heap commands are:

$$\frac{x \notin \text{fv}(e_1) \cup \text{fv}(e_2)}{\{e_1 \hookrightarrow e_2\} x := *e_1 \{[x = e_2] \star e_1 \hookrightarrow e_2\}} \text{LOAD} \quad \frac{}{\{e_1 \hookrightarrow -\} *e_1 := e_2 \{e_1 \hookrightarrow e_2\}} \text{STORE}$$

$$\frac{}{\{e \hookrightarrow -\} \mathbf{free}(e) \{\mathbf{emp}\}} \text{FREE} \quad \frac{x \notin \text{fv}(e)}{\{\mathbf{emp}\} x := \mathbf{new}(e) \{x \hookrightarrow e\}} \text{ALLOC}$$

In addition to the usual rules of Hoare logic for **skip**, assignment, sequencing, and conditionals, we will use the following rules:

$$\frac{\vdash \{H\} c \{J\} \quad \text{fv}(K) \cap \text{mod}(c) = \emptyset}{\vdash \{H \star K\} c \{J \star K\}} \text{FRAME} \quad \frac{\vdash \{[b] \star I\} c \{I\}}{\vdash \{I\} \mathbf{while } b \mathbf{ do } c \{[\neg b] \star I\}} \text{WHILE}$$

$$\frac{\models H \Rightarrow H' \quad \vdash \{H'\} c \{J'\} \quad \models J' \Rightarrow J}{\vdash \{H\} c \{J\}} \text{CONSEQUENCE}$$

Note that the loop test is combined with the invariant using $\star$: $[b] \star I$ says that b holds and I describes the heap. Writing $[b] \bowtie I$ instead would wrongly require the heap to be empty.

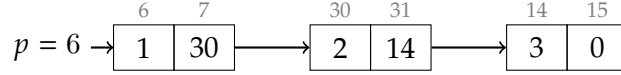
In proofs, it will often be convenient to have structural rules that allow us to rearrange assertions in various ways. The EXISTS rule lets us reason about a command under an existential quantifier: if c takes H to J for every value of the logical variable i , then it takes $\exists i. H$ to $\exists i. J$.

$$\frac{\vdash \{H\} c \{J\}}{\vdash \{\exists i. H\} c \{\exists i. J\}} \text{EXISTS}$$

In the examples, we will use this rule constantly, because unfolding a recursive predicate will always introduce existentials.

2 Linked Lists

We can represent a linked list as a sequence of two-cell *nodes*: the node at address p holds its value at p and the address of the next node at $p + 1$. We use 0 as the null pointer; recall that $0 \notin \text{Addr}$, so no cell is ever stored at address 0, and $0 \hookrightarrow v$ is unsatisfiable. For example, the following heap holds the list $[1, 2, 3]$ at address 6:



As a finite map, this heap is $\{6 \mapsto 1, 7 \mapsto 30, 30 \mapsto 2, 31 \mapsto 14, 14 \mapsto 3, 15 \mapsto 0\}$. Nodes need not be adjacent or in order; only the pointers matter.

A list predicate. We define a heap predicate $\text{list}(p, L)$, meaning “ p points to a list holding the mathematical list L ,” by recursion on L :

$$\begin{aligned} \text{list}(p, []) &\triangleq [p = 0] \\ \text{list}(p, v :: L) &\triangleq \exists q. p \hookrightarrow v \star (p + 1) \hookrightarrow q \star \text{list}(q, L) \end{aligned}$$

Here L is a logical variable ranging over finite lists of integers. The heap above satisfies $\text{list}(6, [1, 2, 3])$: unfolding the definition three times gives

$$\exists q_1, q_2, q_3. 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star q_1 \hookrightarrow 2 \star (q_1 + 1) \hookrightarrow q_2 \star q_2 \hookrightarrow 3 \star (q_2 + 1) \hookrightarrow q_3 \star [q_3 = 0],$$

which holds with $q_1 = 30$, $q_2 = 14$, and $q_3 = 0$.

Because each unfolding claims *new* cells, disjoint from all the others, and the list must end at 0, no L makes $\text{list}(p, L)$ describe a cyclic structure. Similarly, $\text{list}(p_1, L_1) \star \text{list}(p_2, L_2)$ guarantees that the two lists share no nodes. In Hoare logic we would need explicit “acyclic” and “disjoint” predicates to say these things; here $\star$ provides both.

3 Example: Disposing a List

The following program *dispose* frees every node of the list at p :

```

while ( $p \neq 0$ ) do (
   $q := *(p + 1);$ 
  free( $p$ );
  free( $p + 1$ );
   $p := q$ 
)
  
```

Its specification is $\{\text{list}(p, L)\} \text{dispose} \{\text{emp}\}$, and a suitable loop invariant is $\exists L'. \text{list}(p, L')$: the heap always holds exactly the rest of the list. The precondition implies the invariant (take $L' = L$). For

the loop body, the WHILE rule requires us to show $\{[p \neq 0] \star \exists L'. \text{list}(p, L')\} \text{ body } \{\exists L'. \text{list}(p, L')\}$:

$$\begin{array}{ll}
\{(\exists L'. \text{list}(p, L')) \star [p \neq 0]\} \Rightarrow & (\text{unfold}) \\
\{\exists v, y, L''. p \hookrightarrow v \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\} & \\
q := *(p+1); & (\text{LOAD, FRAME}) \\
\{\exists v, y, L''. [q = y] \star p \hookrightarrow v \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\} & \\
\text{free}(p); & (\text{FREE, FRAME}) \\
\{\exists y, L''. [q = y] \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\} & \\
\text{free}(p+1); & (\text{FREE, FRAME}) \\
\{\exists y, L''. [q = y] \star \text{list}(y, L'')\} \Rightarrow & \\
\{\exists L'. \text{list}(q, L')\} & \\
p := q & (\text{assignment}) \\
\{\exists L'. \text{list}(p, L')\} &
\end{array}$$

The first implication unfolds the list: since $p \neq 0$, the list cannot be empty (as $\text{list}(p, [])$ would force $p = 0$), so $L' = v :: L''$ for some v and L'' . Each command is then verified under the existential using EXISTS, and with FRAME to carry along the cells it does not touch. For example, the load uses the LOAD axiom on the single cell $(p+1) \hookrightarrow y$ and frames $p \hookrightarrow v \star \text{list}(y, L'')$; its side condition holds because q occurs in neither $p+1$ nor y . After $\text{free}(p)$, no cell mentions v any more, so we drop $\exists v$ by CONSEQUENCE. The last implication repackages what remains as the invariant with q in place of p , and the assignment axiom then gives the invariant itself.

When the loop exits, we have $(\exists L'. \text{list}(p, L')) \star [p = 0]$. Since $0 \hookrightarrow v$ is unsatisfiable, $\text{list}(0, L')$ forces $L' = []$, and so the heap is empty, as required. Because triples are partial correctness statements, we did not need to prove that the loop terminates (although it does, since the length of L' decreases on every iteration).

4 List Segments

For loops that traverse a list without destroying it, we need to describe the part of the list that has already been visited. A *list segment* from p to q holding L is a list that ends at q instead of at 0:

$$\begin{aligned}
\text{lseg}(p, q, []) &\triangleq [p = q] \\
\text{lseg}(p, q, v :: L) &\triangleq \exists r. p \hookrightarrow v \star (p+1) \hookrightarrow r \star \text{lseg}(r, q, L)
\end{aligned}$$

The following facts are each proved by induction on L :

$$\begin{aligned}
\text{list}(p, L) &\equiv \text{lseg}(p, 0, L) \\
\text{lseg}(p, x, L) \star x \hookrightarrow v \star (x+1) \hookrightarrow y &\Rightarrow \text{lseg}(p, y, L ++ [v]) & (\text{snoc}) \\
\text{lseg}(p, y, L) \star \text{list}(y, L') &\Rightarrow \text{list}(p, L ++ L') & (\text{concat})
\end{aligned}$$

Snoc extends a segment by one node at its end, and concat attaches a list to the end of a segment.

5 Example: List Append

The following program *append*, appends the list at p_2 to the list at p_1 , leaving the result at r .

```

 $x := p_1;$ 
 $n := *(x + 1);$ 
while ( $n \neq 0$ ) do (
   $x := n;$ 
   $n := *(x + 1)$ 
);
 $*(x + 1) := p_2;$ 
 $r := p_1$ 

```

In the program, the variable x always points at a node of the first list, and n holds that node's next pointer; the loop walks x to the last node, and the final store overwrites its null pointer with p_2 . For example, suppose the heap initially holds the lists $[1, 2]$ at address 6 and $[3, 4]$ at address 99:

address	6	7	22	23	46	47	99	100
before	1	46	4	0	2	0	3	22
after	1	46	4	0	2	99	3	22

with $p_1 = 6$ and $p_2 = 99$. Afterwards, $x = 46$ (the last node of the first list), $n = 0$, $r = 6$, and the heap holds the list $[1, 2, 3, 4]$ at r . Only one cell changes—the last node is modified from 0 to p_2 .

The specification for this program

$$\{list(p_1, L_1) \star list(p_2, L_2) \star [p_1 \neq 0]\} \text{ append } \{list(r, L_1 \uplus L_2)\}.$$

Two aspects of this specification deserve comment. First, the pure fact $p_1 \neq 0$ is essential: if the first list is empty, then $x = 0$, so the program accesses address 1, which need not be allocated, and it may produce a memory error. Second, $list(p_2, L_2)$ does not appear in the postcondition because its cells have become part of $list(r, L_1 \uplus L_2)$. The precondition's $\star$ guarantees that the two lists are disjoint, so appending cannot create a cycle.

To keep the proof readable, we abbreviate a node as

$$p:\langle v|q \rangle \triangleq p \hookrightarrow v \star (p+1) \hookrightarrow q.$$

The proof has three parts: the code before the loop, which establishes the invariant; the loop body, which preserves it; and the code after the loop, which establishes the postcondition.

Before the loop. We frame $list(p_2, L_2)$, which this code does not touch. Since $p_1 \neq 0$, the first list is not empty, so we can unfold its first node:

$$\begin{aligned}
& \{list(p_1, L_1) \star [p_1 \neq 0]\} \Rightarrow \\
& \{\exists v, y, L_b. [L_1 = v :: L_b] \star p_1:\langle v|y \rangle \star list(y, L_b)\} \\
& x := p_1; \\
& \{\exists v, y, L_b. [L_1 = v :: L_b] \star [x = p_1] \star x:\langle v|y \rangle \star list(y, L_b)\} \\
& n := *(x + 1); \\
& \{\exists v, y, L_b. [L_1 = v :: L_b] \star [x = p_1] \star [n = y] \star x:\langle v|y \rangle \star list(y, L_b)\} \Rightarrow \\
& \{\exists v, L_b. [L_1 = [] \uplus v :: L_b] \star lseg(p_1, x, []) \star x:\langle v|n \rangle \star list(n, L_b)\}
\end{aligned}$$

Note that the pure fact $[L_1 = v :: L_b]$ must be carried through every assertion: it is the only connection between the logical variables v and L_b and the list L_1 in the specification. In the last step we use $lseg(p_1, x, []) \equiv [p_1 = x]$ and replace y by n . Generalizing $[]$ to an arbitrary L_a , and adding back the frame, suggests the loop invariant

$$I \triangleq \exists L_a, v, L_b. [L_1 = L_a ++ v :: L_b] \star lseg(p_1, x, L_a) \star x:\langle v | n \rangle \star list(n, L_b) \star list(p_2, L_2).$$

The visited part of the first list is the segment from p_1 to x ; x is the current node, holding v and pointing to n ; and the rest of the first list is at n . The assertion we derived is exactly I with $L_a = []$, so the code before the loop establishes the invariant.

The loop body. We must show $\{I \star [n \neq 0]\} x := n; n := *(x + 1) \{I\}$. We again frame $list(p_2, L_2)$. Since $n \neq 0$, the rest of the list is not empty, so we unfold $L_b = w :: L_c$:

$$\begin{aligned} \{I \star [n \neq 0]\} &\Rightarrow && \text{(unfold)} \\ \{\exists L_a, v, w, m, L_c. [L_1 = L_a ++ v :: w :: L_c] \\ &\quad \star lseg(p_1, x, L_a) \star x:\langle v | n \rangle \star n:\langle w | m \rangle \star list(m, L_c)\} &\Rightarrow & \text{(snoc)} \\ \{\exists L_a, v, w, m, L_c. [L_1 = L_a ++ v :: w :: L_c] \\ &\quad \star lseg(p_1, n, L_a ++ [v]) \star n:\langle w | m \rangle \star list(m, L_c)\} \\ x := n; &&& \text{(assignment)} \\ \{\exists L_a, v, w, m, L_c. [L_1 = L_a ++ v :: w :: L_c] \\ &\quad \star lseg(p_1, x, L_a ++ [v]) \star x:\langle w | m \rangle \star list(m, L_c)\} \\ n := *(x + 1) &&& \text{(LOAD, FRAME)} \\ \{\exists L_a, v, w, m, L_c. [L_1 = L_a ++ v :: w :: L_c] \star [n = m] \\ &\quad \star lseg(p_1, x, L_a ++ [v]) \star x:\langle w | m \rangle \star list(m, L_c)\} \Rightarrow \\ \{I\} \end{aligned}$$

The snoc step folds the current node into the visited segment. The final implication re-establishes I with $L_a ++ [v]$ for L_a , w for v , and L_c for L_b . This fact holds because $L_a ++ v :: w :: L_c = (L_a ++ [v]) ++ w :: L_c$, and $x:\langle w | m \rangle \star list(m, L_c)$ becomes $x:\langle w | n \rangle \star list(n, L_c)$ because $n = m$.

After the loop. When the loop exits, we have $I \star [n = 0]$. Since $n = 0$, the rest of the list must be empty, so $L_b = []$ and hence $L_1 = L_a ++ [v]$:

$$\begin{aligned} \{I \star [n = 0]\} &\Rightarrow \\ \{\exists L_a, v. [L_1 = L_a ++ [v]] \star lseg(p_1, x, L_a) \star x:\langle v | 0 \rangle \star list(p_2, L_2)\} \\ *(x + 1) := p_2; &&& \text{(STORE, FRAME)} \\ \{\exists L_a, v. [L_1 = L_a ++ [v]] \star lseg(p_1, x, L_a) \star x:\langle v | p_2 \rangle \star list(p_2, L_2)\} &\Rightarrow & \text{(snoc)} \\ \{lseg(p_1, p_2, L_1) \star list(p_2, L_2)\} \\ r := p_1 &&& \text{(assignment)} \\ \{lseg(r, p_2, L_1) \star list(p_2, L_2)\} &\Rightarrow & \text{(concat)} \\ \{list(r, L_1 ++ L_2)\} \end{aligned}$$

This establishes the postcondition and completes the proof.

6 The Magic Wand

In standard Hoare logic, the assignment axiom $\{P[e/x]\} x := e \{P\}$ works *backwards*: it computes the weakest precondition of an assignment by substitution. What is the weakest precondition of a heap store such as $*x := 5$ with respect to a postcondition Q ? Substitution doesn't work, because Q may refer to the cell at x under a different name—e.g., an assertion such as $y \hookrightarrow 3$ may refer to the same cell when x and y are aliases. What we would like to say is: “I hold the cell at x , together with a *promise*: add $x \hookrightarrow 5$, and you get Q .” Our existing assertion language has no way to express such a promise, so we add a new connective, called magic wand.

Definition. The *magic wand*, or separating implication, is defined by

$$(\sigma, h) \models_I H \star J \quad \text{iff} \quad \text{for all } h' \perp h \text{ with } (\sigma, h') \models_I H, \text{ we have } (\sigma, h \uplus h') \models_I J.$$

Intuitively, $H \star J$ describes a heap that becomes a J -heap when you add an H -heap; it is sometimes described as “ J with an H -shaped hole.” The wand is to $\star$ as implication is to conjunction. In particular, it satisfies modus ponens and a currying law:

$$H \star (H \rightarrow J) \Rightarrow J \qquad (K \star H \Rightarrow J) \text{ iff } (K \Rightarrow H \rightarrow J)$$

For example, $\text{emp} \rightarrow H \equiv H$, and any heap satisfying $y \hookrightarrow 4$ also satisfies

$$x \hookrightarrow 3 \rightarrow (x \hookrightarrow 3 \star y \hookrightarrow 4).$$

(If x and y are aliases, no disjoint heap satisfies $x \hookrightarrow 3$, so the wand holds vacuously.)

Weakest preconditions for heap commands. With the wand, we can state the weakest preconditions of the heap commands, where “weakest” is with respect to our fault-avoiding triples:

Command	Weakest precondition for Q
$*e_1 := e_2$	$(e_1 \hookrightarrow -) \star ((e_1 \hookrightarrow e_2) \rightarrow Q)$
$\text{free}(e)$	$(e \hookrightarrow -) \star Q$
$x := \text{new}(e)$	$\forall i. (i \hookrightarrow e) \rightarrow Q[i/x] \quad (i \text{ fresh})$

For a store, we must own the cell at e_1 (so that the store does not fault), and the rest of the heap must turn into a Q -heap once a cell $e_1 \hookrightarrow e_2$ is added. For allocation, the rest of the heap must turn into a Q -heap once *any* new cell $i \hookrightarrow e$ is added, with x holding its address i , because NEW may choose any unused address. For example, with $Q = x \hookrightarrow 5 \star y \hookrightarrow 7$,

$$wp(*x := 5, Q) = x \hookrightarrow - \star (x \hookrightarrow 5 \rightarrow (x \hookrightarrow 5 \star y \hookrightarrow 7)) \Leftarrow x \hookrightarrow - \star y \hookrightarrow 7,$$

by currying, since $y \hookrightarrow 7 \star x \hookrightarrow 5 \Rightarrow x \hookrightarrow 5 \star y \hookrightarrow 7$. This is exactly what we would expect.

Backward rules from forward ones. We do not need new proof rules to use these weakest preconditions. For example, the backward rule for stores follows from the forward STORE axiom. Let $K = (e_1 \hookrightarrow e_2) \rightarrow Q$. Since $\text{mod}(*e_1 := e_2) = \emptyset$, the STORE axiom and FRAME give

$$\{e_1 \hookrightarrow - \star K\} *e_1 := e_2 \{e_1 \hookrightarrow e_2 \star K\}.$$

By modus ponens, $e_1 \hookrightarrow e_2 \star ((e_1 \hookrightarrow e_2) \multimap Q) \Rightarrow Q$, so by CONSEQUENCE,

$$\{(e_1 \hookrightarrow -) \star ((e_1 \hookrightarrow e_2) \multimap Q)\} * e_1 := e_2 \{Q\}.$$

In other words, the forward small axioms together with FRAME already suffice; the wand simply lets us *state* weakest preconditions.

Where the magic wand shows up. The wand plays a central role in modern program logics. In Iris, for example, the rules for weakest preconditions are stated using $\multimap$, much like the backward rules above. Viper supports magic wands in specifications. The wand is also useful for describing partial data structures: for instance, $\text{list}(x, L_2) \multimap \text{list}(p, L_1 \uplus L_2)$ describes a list with a hole, which becomes a complete list once the list at x is supplied. On the other hand, the wand is notoriously hard to automate, which is one reason why most tools reason in the forward direction, often using symbolic execution.

Further Reading

Reynolds's [lecture notes](#) cover list segments, the structural rules (including the form of the EXISTS rule used here), and backward reasoning with the magic wand. Charguéraud's textbook *Separation Logic Foundations* (Software Foundations, Volume 6) takes as primitive a variant of the EXISTS rule with the existential only in the precondition (`triple_hexists`), and derives the form used here from it (`triple_hexists2`).